Stage 2 Implementation Report

MealPlanner AI - an AI agent for weekly meal planning

Suleyman Kavas | EECS3311 Software Design | Fall 2026 | Individual project

Submitted: November 2026 | Implementation period: October 20 - November 10, 2026

Repository: https://github.com/esadkavas/mealplanner-ai

Stage 1 design: `MealPlanner_Stage1_Design.docx` (same repository).

Report map

Section	Contents
1	Project overview, implementation status and limitations
2	Feature to implementation mapping (F01-F10): classes, methods, patterns, collaboration, videos
3	Implemented design patterns and updated UML class and sequence diagrams
4	Changes from Stage 1 and their rationale
5	AI tools and collaboration log
6	Development and verification summary: weekly commits, checks, defects, limitations
7	Demonstration index (video links by feature) and references

1 Project overview and implementation status

MealPlanner AI helps one home cook decide what to cook during the week and what to buy. The user records dietary preferences, pantry stock and structured recipes. An AI agent then recommends recipes, builds meal plans for 1 to 7 days, revises saved plans from natural-language requests, and proposes ingredient substitutions. Java code scales recipes and totals shopping lists. The application is a Java 21 program with a **Swing GUI** and an **interactive CLI**; both call the same `PlannerFacade`. Data is stored locally in one JSON file.

AI/LLM integration. The application calls Google **Gemini 3.5 Flash-Lite** (model ID `gemini-3.5-flash-lite`, verified as a stable model on the official model page on 6 November 2026) through the Gemini REST API. The model is not used as a chatbot. `AgentCoordinator` runs a bounded agent loop in which the model must call validated local tools (search stored recipes, read the profile, pantry or a saved plan, list catalogue ingredients, and check a candidate plan) before producing a JSON answer that follows a schema. Java then validates every recipe ID, ingredient, quantity, slot and dietary constraint independently, and anything that would change saved data is shown as a preview that the user must accept.

Implementation status

Area	Status
Features F01-F10	All ten Stage 1 features are implemented in both the GUI and the CLI. No feature was removed, replaced or added (Section 2).
Design patterns	Facade, Adapter, Decorator, Builder and Prototype are implemented as designed (Section 3).
LLM / agent	Gemini adapter, logging decorator, bounded coordinator (max 5 tool steps + 1 final call, 15 s per call, 90 s total), 6 tools, schema-checked answers, Java validation, preview/accept.
Persistence	Atomic JSON saves, stale-proposal rejection, single-process data-directory lock, invalid stored data reported instead of replaced.
Verification	82 JUnit tests pass on every implementation commit; end-to-end CLI and GUI runs (Section 6). Live Gemini check: passed on 8 November 2026 on the student's Mac (API key, function calling, conversation replay and structured JSON output; 315 input + 26 output tokens).

Limitations and incomplete functionality

- **Live AI quality.** In live testing on 8 November 2026, all four AI features (F05-F08) produced answers that passed Java validation on the first attempt, and the agent used its tools as intended (for example `checkPlan` returned valid before the plan was proposed). The free tier allows only a few requests per minute: four AI actions in a row hit HTTP 429, which led to the automatic one-time retry described in Section 6.3. Answers can differ between runs. A weak answer is rejected with an explanation, and the user can retry.
- The AI part depends on an external service. Without a key, or when the service is down or rate-limited, AI actions fail with a clear message and the deterministic features (F01-F04, F09, F10) keep working. No hard-coded AI answers are ever substituted.
- Scope is the Stage 1 scope: one local user, a fixed catalogue of 50 ingredients with one canonical unit each, 21 starter recipes, and no unit conversion, nutrition, prices, recipe deletion or editing of saved plans in place.
- Diet eligibility uses catalogue tags only. It is not a medical allergy or religious-compliance check.

- Stage 2 includes basic verification only. The full deterministic test suite and the KUMA agent-validation tests are Stage 3 work.
- The repository has no commits in reporting periods 1 (Oct 20-26) and 2 (Oct 27-Nov 2). The one-time 5-point deduction therefore applies (Section 6.1).

2 Feature to implementation mapping

Every feature is reached through `PlannerFacade`. The GUI calls the facade through `PlannerGUI.submit()`, which runs the call on a `SwingWorker` background thread. The CLI calls it from `PlannerCLI.execute()`. Both display results with the shared `TextFormat` class. Changes are applied to a deep copy of `AppState`, saved atomically by `JsonStateStore`, and only then published. Section 3 describes the patterns named below.

F01 - Manage dietary preferences

Feature ID and name	F01 - Manage dietary preferences
Stage 1 feature/reference	F01 (UC01, SD01)
Status or change	Implemented (as designed)
Classification	Deterministic
Implemented classes/files and responsibilities	`PreferencesPanel` (GUI form) and `PlannerCLI.profile()` (CLI `profile set`) collect raw input. `ProfileInput` holds the text exactly as typed. `ConstraintValidator` validates the fields and builds the profile. `PreferenceProfile` is the immutable saved profile. `PlannerFacade` copies, saves and publishes the state. `JsonStateStore` writes the file atomically.
Key methods	`PreferencesPanel.save()`, `PlannerFacade.updateProfile()`, `ConstraintValidator.profile(ProfileInput)`, `ConstraintValidator.toProfile()`, `AppState.deepCopy()`, `JsonStateStore.save()`
Design patterns	Facade; Prototype (state copy)
Video link/timestamp	GUI: Video 1 (GUI) 0:00 CLI: Video 2 (CLI) 0:46

How the classes collaborate. The interface wraps the diet, excluded IDs, servings and time limit in a `ProfileInput` without converting numbers. `ConstraintValidator.profile()` returns field-level errors (unsupported diet, unknown ingredient ID, servings outside 1-12, minutes outside 1-240) that the GUI shows in red and the CLI prints. If the input is valid, the facade converts it to an immutable `PreferenceProfile`, applies it to a deep copy of `AppState`, increments the revision and saves. The active state changes only after the save succeeds. Search, recommendations, plans and substitutions all read this profile afterwards.

F02 - Manage pantry inventory

Feature ID and name	F02 - Manage pantry inventory
Stage 1 feature/reference	F02 (UC02, SD01)

Status or change	Implemented (as designed)
Classification	Deterministic
Implemented classes/files and responsibilities	`PantryPanel` (table, ingredient selector, quantity field, Remove) and `PlannerCLI.pantry()` collect input. `PantryInput` represents SET or REMOVE. `ConstraintValidator.pantry()` checks the ingredient ID and quantity. `Catalogue` supplies canonical units. `AppState` stores the pantry map.
Key methods	`PlannerFacade.updatePantry()`, `ConstraintValidator.pantry()`, `AppState.setPantryQuantity()`, `AppState.removePantryItem()`, `JsonStateStore.save()`
Design patterns	Facade; Prototype (state copy)
Video link/timestamp	GUI: Video 1 (GUI) 1:10 CLI: Video 2 (CLI) 1:00

How the classes collaborate. The interface sends a `PantryInput`. The validator rejects unknown IDs, malformed numbers and negative quantities. Zero is stored and labelled as unavailable. The unit always comes from the catalogue, so one ingredient is never measured in two units. The facade updates a copied state and saves it, or reports the storage error and keeps the previous pantry. Creating plans never changes pantry stock.

F03 - Create and import structured recipes

Feature ID and name	F03 - Create and import structured recipes
Stage 1 feature/reference	F03 (UC03, SD10, SD08)
Status or change	Implemented (as designed)
Classification	Deterministic
Implemented classes/files and responsibilities	`RecipeFormDialog` (GUI form), `RecipesPanel.importJson()` (file chooser) and `PlannerCLI recipe create/import` collect input. `RecipeDocument` is the shared form/JSON structure. `RecipeService` parses and normalises recipes. `ConstraintValidator.recipe()` separates structural errors from eligibility. `Proposal` (IMPORT_RECIPE) holds the preview. `PlannerFacade.accept()` saves it.
Key methods	`PlannerFacade.previewRecipe()`, `previewRecipeJson()`, `previewRecipeFile()`, `RecipeService.parse()`, `RecipeService.fromDocument()`, `ConstraintValidator.recipe()`, `Proposal.validate()`, `PlannerFacade.accept()`
Design patterns	Facade; Prototype (candidate state)
Video link/timestamp	GUI: Video 1 (GUI) 2:00 CLI: Video 2 (CLI) 1:10

How the classes collaborate. File imports are size-checked (100 KB) and parsed strictly: malformed JSON, unknown fields and missing fields are rejected and nothing is imported. `ConstraintValidator.recipe()` reports structural errors (unknown ingredient, non-positive quantity, duplicate ingredient, ranges, empty steps), which block the preview. Eligibility conflicts become a visible "ineligible" warning instead. A valid recipe gets an ID and is placed in a candidate

copy of the state inside a `Proposal`. Only `accept()` saves it, after re-checking that the proposal is current and still valid.

F04 - Search and filter recipes

Feature ID and name	F04 - Search and filter recipes
Stage 1 feature/reference	F04 (UC04, SD02)
Status or change	Implemented (as designed)
Classification	Deterministic
Implemented classes/files and responsibilities	`RecipesPanel.search()` and `PlannerCLI recipe search` build a `SearchFilter` (title text, ingredient IDs, max minutes, pantry-only). `RecipeService.search()` applies the profile and the filters. `ConstraintValidator.isEligible()` checks eligibility. The same search is exposed to the agent as the `searchRecipes` tool.
Key methods	`PlannerFacade.search()`, `RecipeService.validateFilter()`, `RecipeService.search()`, `RecipeService.shortages()`, `ConstraintValidator.isEligible()`
Design patterns	Facade
Video link/timestamp	GUI: Video 1 (GUI) 4:18 CLI: Video 2 (CLI) 1:30

How the classes collaborate. The filter is validated (known ingredient IDs, minutes 1-240). Every stored recipe is first checked against the saved profile, so diet, exclusions and the time limit always apply. A filter time limit can only tighten the saved limit. The title, ingredient and pantry-only filters follow, and pantry-only compares quantities at base servings. An empty result gives a message telling the user to change the filters. Constraints are never relaxed and nothing is saved.

F05 - Recommend recipes from preferences and pantry

Feature ID and name	F05 - Recommend recipes from preferences and pantry
Stage 1 feature/reference	F05 (UC05, SD03, SD09, SD11)
Status or change	Implemented (as designed)
Classification	Hybrid (AI-based ranking and explanation, Java validation and shortages)
Implemented classes/files and responsibilities	`RecommendPanel` and `PlannerCLI agent recommend` collect the request. `PlannerFacade.recommend()` coordinates the request. `AgentCoordinator` runs the bounded loop. `ToolRegistry` provides readProfile, readPantry, searchRecipes and listIngredients. `ToolScope` holds the read-only snapshot and the retrieved IDs. `LoggingLLMClient` wraps `GeminiAdapter`. `AgentPrompts` supplies the task prompt and schema. `Drafts` checks the answer against the schema. `RecipeService.recommendations()` validates the result and computes shortages. `RecommendationList` is the output.
Key methods	`PlannerFacade.recommend()`, `AgentCoordinator.run()`, `LLMClient.nextStep()`, `ToolRegistry.execute()`, `RecipeService.search()`, `LLMClient.finish()`,

	`Drafts.status()`, `Drafts.recommendation()`, `RecipeService.recommendations()`
Design patterns	Facade, Adapter, Decorator
Video link/timestamp	GUI: Video 1 (GUI) 3:08 CLI: Video 2 (CLI) 1:44

How the classes collaborate. The facade takes a deep-copy snapshot and starts the agent with the request (at most 2,000 characters). The model typically reads the pantry and calls `searchRecipes` with filters it derived from the request; each call is schema-checked and executed against the snapshot. The final JSON lists up to five recipe IDs with reasons and the constraints the model took from the request (time limit, required or avoided ingredients). `RecipeService.recommendations()` then rejects the whole answer if any ID is unknown, duplicated, not returned by a tool, ineligible, or violates those constraints. Java computes the missing quantities. NO_MATCH, NEEDS_CLARIFICATION and UNSUPPORTED answers are shown to the user. Nothing is saved.

F06 - Generate a meal plan

Feature ID and name	F06 - Generate a meal plan
Stage 1 feature/reference	F06 (UC06, SD03, SD04, SD08, SD09)
Status or change	Implemented (as designed)
Classification	Hybrid (AI selects recipes; Java fixes the plan shape, builds and validates)
Implemented classes/files and responsibilities	`PlansPanel.generate()` and `PlannerCLI agent plan` build a `PlanRequest` (start date, 1-7 days, slots, servings, no-repeat). `PlanService` validates, checks feasibility, and directs `MealPlanBuilder`. The `checkPlan` tool lets the agent test a selection. `Proposal` (PLAN) is accepted through `PlannerFacade.accept()`.
Key methods	`PlannerFacade.createPlan()`, `PlanService.validateRequest()`, `PlanService.checkFeasible()`, `AgentCoordinator.run()`, `PlanService.fromDraft()`, `MealPlanBuilder.start()`, `addMeal()`, `build()`, `PlanService.validate()`, `PlannerFacade.accept()`
Design patterns	Facade, Adapter, Decorator, Builder, Prototype (recipe snapshots)
Video link/timestamp	GUI: Video 1 (GUI) 6:46 CLI: Video 2 (CLI) 2:30

How the classes collaborate. The interface fixes the plan shape before any model call. `PlanService.checkFeasible()` rejects impossible requests immediately, for example a no-repeat plan that needs more different recipes than are eligible, so the model is never asked to try. The agent searches recipes and is prompted to call `checkPlan` with its full selection, which runs the real `PlanService.fromDraft()` and returns problems for it to fix. The final entries contain only day, slot and recipe ID. `PlanService.fromDraft()` resolves each ID, requires that it was retrieved by a tool, and directs `MealPlanBuilder`. The builder deep-copies a recipe snapshot into each `MealEntry`, rejects duplicate, unrequested or resized slots, and builds only a complete plan. Full validation (coverage, eligibility, no-repeat) runs before the preview is shown. Accepting saves a new plan.

F07 - Revise a plan through natural language

Feature ID and name	F07 - Revise a plan through natural language

Stage 1 feature/reference	F07 (UC07, SD03, SD05, SD08, SD09)
Status or change	Implemented (as designed)
Classification	Hybrid
Implemented classes/files and responsibilities	`PlansPanel.revise()` and `PlannerCLI agent revise` collect the request. `PlannerFacade.revisePlan()` coordinates the revision. `ToolRegistry` provides readPlan (weekday, date, current eligibility), searchRecipes and checkPlan. `PlanService.revise()` and `PlanService.diff()` produce the revised plan and the change list. `MealPlan.deepCopy()` protects the original. `Proposal` stores the slot changes.
Key methods	`PlannerFacade.revisePlan()`, `AgentCoordinator.run()`, `PlanService.revise()`, `MealPlan.deepCopy()`, `MealPlanBuilder.addMeal()`, `build()`, `PlanService.validate()`, `PlanService.diff()`, `PlannerFacade.accept()`
Design patterns	Facade, Adapter, Decorator, Prototype, Builder
Video link/timestamp	GUI: Video 1 (GUI) 8:02 CLI: Video 2 (CLI) 3:06

How the classes collaborate. The task prompt maps each plan day to its weekday and date, so "Tuesday dinner" can be resolved. The agent reads the plan, searches for replacement recipes and returns only the replaced slots. `PlanService.revise()` deep-copies the saved plan and rebuilds it with `MealPlanBuilder`, replacing only those slots and keeping dates, servings, no-repeat and every other meal. The whole revised plan, including unchanged meals, is validated against the current preferences. The preview lists each changed slot as before and after. An accepted revision is saved under a new plan ID that records `derivedFrom`, and the source plan is never overwritten. A request naming a slot that is not in the plan leads to a clarification answer or a validation error.

F08 - Suggest and apply ingredient substitutions

Feature ID and name	F08 - Suggest and apply ingredient substitutions
Stage 1 feature/reference	F08 (UC08, SD03, SD06, SD08, SD09)
Status or change	Implemented (as designed)
Classification	Hybrid
Implemented classes/files and responsibilities	`RecipesPanel.substitute()` (ingredient chooser and request) and `PlannerCLI agent substitute` collect input. `PlannerFacade.substitute()` coordinates the request. `ToolRegistry` provides readProfile and listIngredients. `Drafts.substitution()` turns the answer into a `SubstitutionDraft`. `RecipeService.substitute()` applies it to a copy. `ConstraintValidator.recipe()` checks the variant. `Proposal` (SUBSTITUTE_RECIPE) holds the preview.
Key methods	`PlannerFacade.substitute()`, `AgentCoordinator.run()`, `Drafts.substitution()`, `RecipeService.substitute()`, `Recipe.deepCopy()`, `ConstraintValidator.recipe()`, `PlannerFacade.accept()`
Design patterns	Facade, Adapter, Decorator, Prototype

Video link/timestamp	GUI: Video 1 (GUI) 5:58 CLI: Video 2 (CLI) 3:56

How the classes collaborate. The source recipe, its steps and the selected ingredient go into the task prompt, even if the recipe is currently ineligible. The agent must call `listIngredients`, and the replacement must be one of the ingredient IDs that tool returned. The model proposes a quantity in the replacement's own canonical unit, because no unit conversion is done, plus rewritten steps. `RecipeService.substitute()` applies the draft to a deep copy and changes only the selected ingredient and the listed steps. The full variant must be structurally valid and eligible for the profile. The accepted variant is saved as a new recipe, so the original and existing plan snapshots are unchanged.

F09 - Adjust recipe serving quantities

Feature ID and name	F09 - Adjust recipe serving quantities
Stage 1 feature/reference	F09 (UC09, SD06, SD08)
Status or change	Implemented (as designed)
Classification	Deterministic
Implemented classes/files and responsibilities	`RecipesPanel.scale()` and `PlannerCLI recipe scale` collect the target servings. `RecipeService.scale()` does the arithmetic on a copy. `Recipe.deepCopy()` protects the source. `Proposal` (SCALE_RECIPE) holds the preview. `Quantities` handles display rounding.
Key methods	`PlannerFacade.scale()`, `RecipeService.scale()`, `Recipe.deepCopy()`, `ConstraintValidator.recipe()`, `PlannerFacade.accept()`
Design patterns	Facade, Prototype
Video link/timestamp	GUI: Video 1 (GUI) 5:14 CLI: Video 2 (CLI) 1:54

How the classes collaborate. Target servings must be a whole number from 1 to 12. The service deep-copies the recipe and computes quantity x target / base with `BigDecimal` (DECIMAL128). Display rounds to two decimals without changing stored values, so 200 g x 5/2 = 500 g and 1 onion x 5/3 shows 1.67. Steps and preparation time are kept with a note that cooking time may need judgement. The variant is structurally validated and labelled ineligible if needed. Accepting saves it as a new recipe and leaves the source unchanged.

F10 - Generate a consolidated shopping list

Feature ID and name	F10 - Generate a consolidated shopping list
Stage 1 feature/reference	F10 (UC10, SD07)
Status or change	Implemented (as designed)
Classification	Deterministic
Implemented classes/files and responsibilities	`PlansPanel.shopping()` and `PlannerCLI shopping generate` request the list. `ShoppingService.generate()` totals the plan's recipe snapshots. `ShoppingList` is the output.

	`TextFormat.shopping()` labels it "Total ingredients required".
Key methods	`PlannerFacade.shopping()`, `ShoppingService.generate()`, `MealPlan.getMeals()`, `TextFormat.shopping()`
Design patterns	Facade
Video link/timestamp	GUI: Video 1 (GUI) 7:46 CLI: Video 2 (CLI) 2:54

How the classes collaborate. For every meal, each snapshot quantity is scaled by meal servings / snapshot base servings and summed by canonical ingredient ID with `BigDecimal`. Only the final totals are rounded for display. Because each entry owns its snapshot, later recipe edits cannot change an existing plan's list. Pantry stock is deliberately not deducted, and the output says so. A missing plan or a corrupt snapshot produces an explicit error.

New features: none. All ten Stage 1 features are implemented. The `llm-check` command (`ConnectionCheck`) is a diagnostic for verifying the provider connection and is not counted as a feature.

3 Implemented design patterns

All five Stage 1 patterns are implemented. Each solves a specific problem in working code, as described below.

Facade

Design problem	The GUI and the CLI must offer the same workflows without each one handling validation, agent calls, proposals and saving on its own.
Participants and roles	Facade: `PlannerFacade`. Subsystems: `RecipeService`, `PlanService`, `ShoppingService`, `ConstraintValidator`, `AgentCoordinator`, `StateStore`. Clients: `PlannerGUI` (and its panels) and `PlannerCLI`.
Source files and methods	`facade/PlannerFacade.java`: `updateProfile()`, `updatePantry()`, `previewRecipe*()`, `search()`, `recommend()`, `createPlan()`, `revisePlan()`, `substitute()`, `scale()`, `shopping()`, `accept()`, `reject()`.
How it follows the pattern / why useful	Each public operation hides a multi-step workflow behind one call that returns `Result`. For example, `accept()` checks staleness, re-validates by proposal kind, copies the candidate, increments the revision, saves, publishes the new state and marks the proposal consumed, all under one lock. The two interfaces contain no business rules, so they behave the same (the CLI and GUI tests drive the same facade).
Compared with Stage 1	Matches Stage 1. The facade also exposes `snapshot()` for display and `reject()` for symmetry.

Adapter

Design problem	The Gemini request and response formats (contents, parts, function declarations, thought signatures, usage metadata, HTTP errors) must not spread into the agent logic, and the provider must be replaceable.
Participants and roles	Target: `LLMClient` (`nextStep()`, `finish()`). Adapter: `GeminiAdapter`. Adaptee: the Gemini REST API `models/{model}:generateContent`. Client: `AgentCoordinator`.
Source files and methods	`agent/LLMClient.java`, `llm/GeminiAdapter.java`: `nextStep()`, `finish()`, `contents()`, `post()`, `httpError()`.

How it follows the pattern / why useful	`GeminiAdapter` turns the application's `AgentContext` (task prompt, `ToolSpecs`, transcript) into Gemini JSON, and turns the reply into `AgentStep.ToolStep`, `FinalReady` or `Invalid` plus `TokenUsage`. Gemini 3 models require their thought signatures to be returned unchanged during function calling. The adapter keeps the raw model content as an opaque `providerState` that the coordinator stores but never reads. It also translates the neutral tool and answer schemas into Gemini's uppercase Schema type names, and retries once after HTTP 429 when Google's suggested delay fits within the time limit. HTTP and format failures become `LLMException` kinds with sanitized messages. Tests replace the provider with a local fake HTTP server, and the coordinator with a scripted `LLMClient`.
Compared with Stage 1	Matches Stage 1. The opaque `providerState` and the `Invalid` step type were added once the thought-signature requirement was confirmed in Google's documentation.

Decorator

Design problem	Every model call should be timed and logged (and token use counted) without mixing logging into the coordinator or the provider adapter.
Participants and roles	Component: `LLMClient`. Concrete component: `GeminiAdapter`. Decorator: `LoggingLLMClient` (implements `LLMClient` and holds one wrapped `LLMClient`).
Source files and methods	`llm/LoggingLLMClient.java`: `nextStep()`, `finish()`, `record()`, `usage()`; wired in `app/AppContext.start()`.
How it follows the pattern / why useful	Both methods delegate to the wrapped client, measure the time, and write one sanitized line to `data/logs/agent.log` (task, call, duration, outcome, input/output tokens). They then return the same object or rethrow the same exception. Prompts, request text, tool arguments and keys are never logged. The running totals feed the GUI status bar and the CLI `status` command. The coordinator does not know whether it holds a decorated client.
Compared with Stage 1	Matches Stage 1. Token counts were added to the log so AI usage can be reported.

Builder

Design problem	A meal plan must be complete and consistent (every requested day/slot exactly once, fixed servings, independent snapshots) before anyone can use it. The model must not be able to change the plan's shape.
Participants and roles	Builder: `MealPlanBuilder`. Product: `MealPlan`, whose constructor is package-private, so only the builder can create one. Director: `PlanService` (`fromDraft()`, `revise()`).
Source files and methods	`domain/MealPlanBuilder.java`: `start(PlanRequest)`, `addMeal(day, slot, recipe, servings)`, `derivedFrom()`, `explanation()`, `build()`; `service/PlanService.java`.
How it follows the pattern / why useful	`start()` records the fixed `PlanRequest`. `addMeal()` rejects days outside the plan, unrequested slots, changed servings and duplicates, and deep-copies the recipe snapshot. `build()` throws unless every requested slot is filled. The director turns model output into builder calls, so a malformed AI answer can never yield a partial plan.
Compared with Stage 1	Matches Stage 1. `start()` is a static factory, and two optional steps (`derivedFrom`, `explanation`) were added for revisions.

Prototype

Design problem	Scaling, substitution, plan revision and every save must work on independent copies, so shared mutable lists can never change a stored recipe, a saved plan or the active

	state by accident.
Participants and roles	Prototype interface: explicit `deepCopy()` methods. Concrete prototypes: `Recipe`, `MealEntry`, `MealPlan`, `AppState`. Clients: `RecipeService`, `PlanService`, `MealPlanBuilder`, `PlannerFacade`.
Source files and methods	`Recipe.deepCopy()`, `MealPlan.deepCopy()`, `MealEntry.deepCopy()`, `AppState.deepCopy()`; used in `RecipeService.scale()`, `RecipeService.substitute()`, `PlanService.revise()`, `MealPlanBuilder.addMeal()` and `PlannerFacade.commit()` / `accept()`.
How it follows the pattern / why useful	Each copy duplicates its mutable children: ingredient and step lists, entries and snapshots, and maps. Value objects (`IngredientAmount`, `PreferenceProfile`, `BigDecimal`) are immutable and shared safely. Copies used for variants and revisions get new IDs, with `derivedFrom` pointing to the source. Getters return read-only views. `DomainCopyTest` checks that editing a copy never changes the source.
Compared with Stage 1	Matches Stage 1, and is also used for the whole-state copy-on-write save.

Updated UML diagrams

The class diagram keeps the four Stage 1 views (CD01-CD04) and reflects the submitted code. Classes marked <<new in Stage 2>> were added during implementation (see Section 4). The PlantUML sources are in `docs/uml/`. The use-case diagram (Stage 1 Section 7) is unchanged because no feature or actor changed. SD03 and SD09 were updated because the agent loop and provider calls gained details. The other sequence diagrams still describe the implementation.

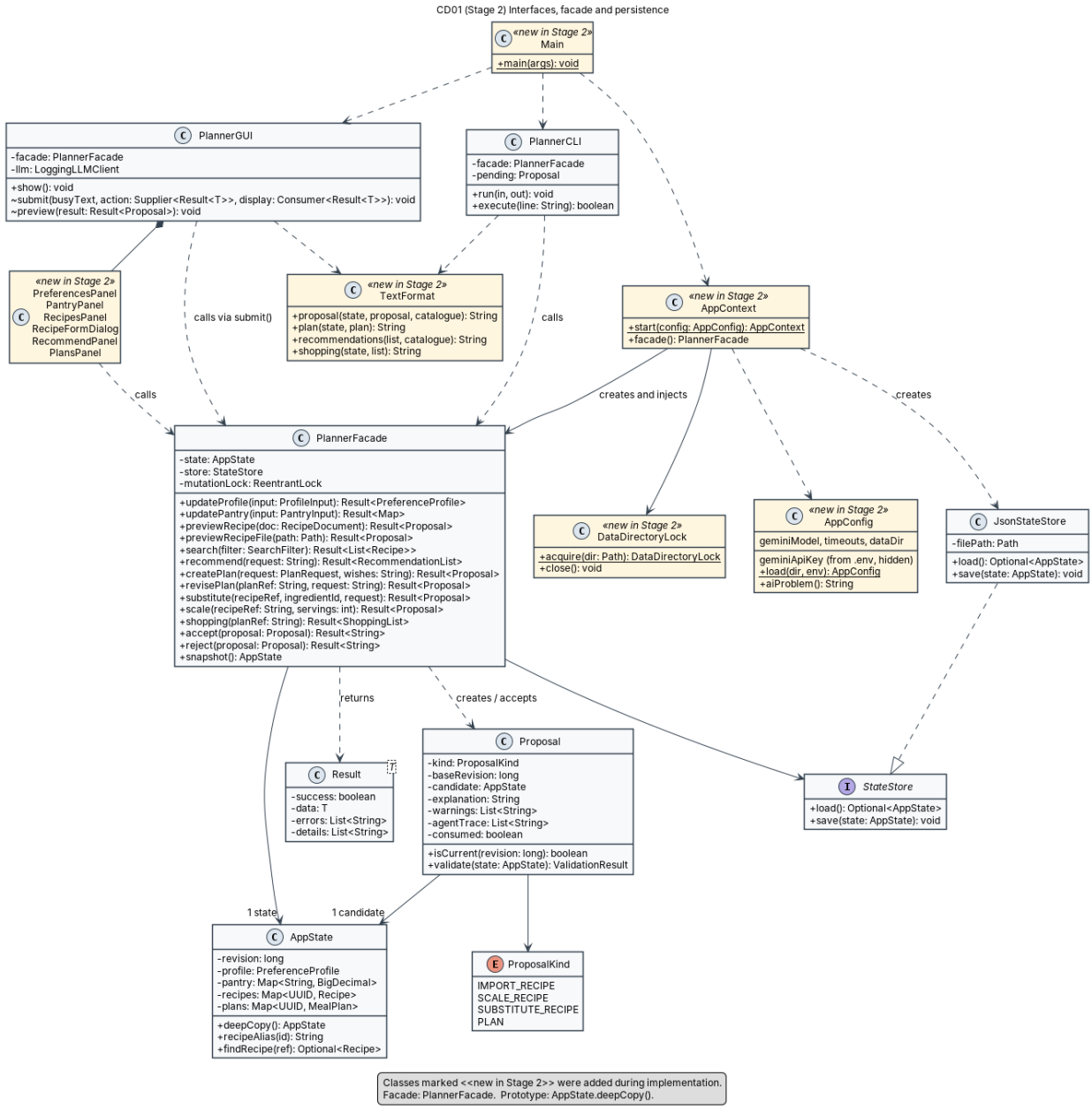


Figure 1. CD01: interfaces, facade and persistence.

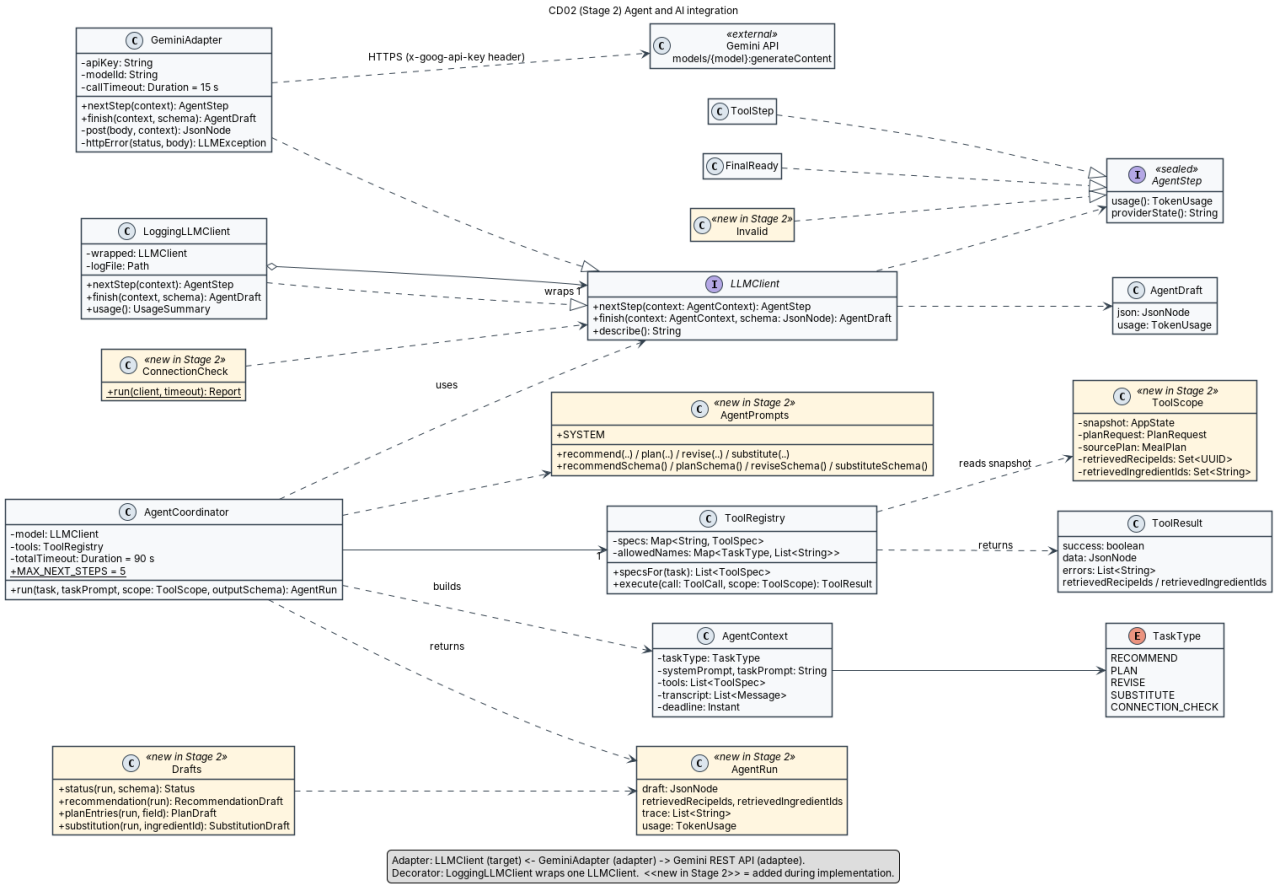


Figure 2. CD02: agent and AI integration (Adapter, Decorator).

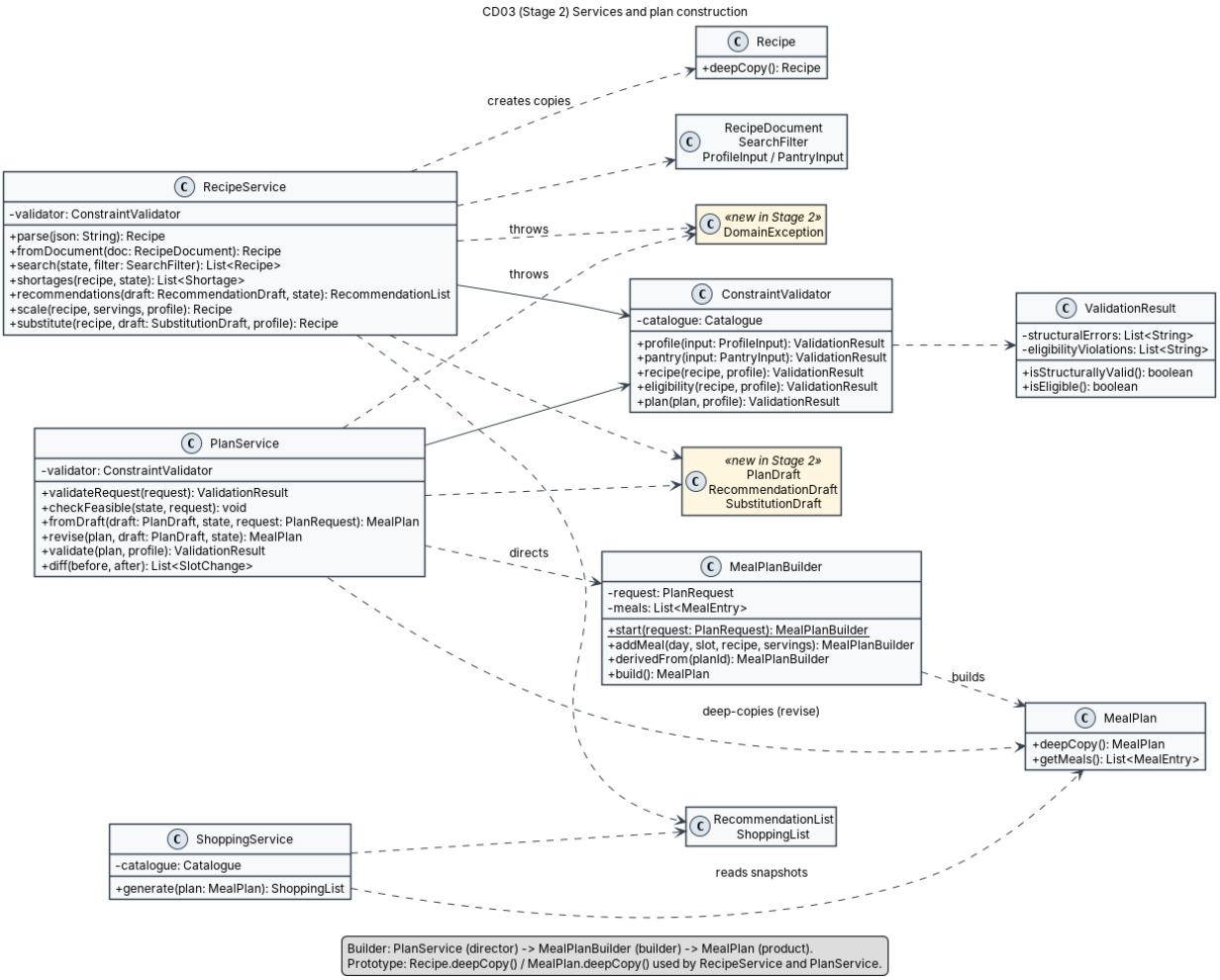


Figure 3. CD03: services and plan construction (Builder, Prototype).

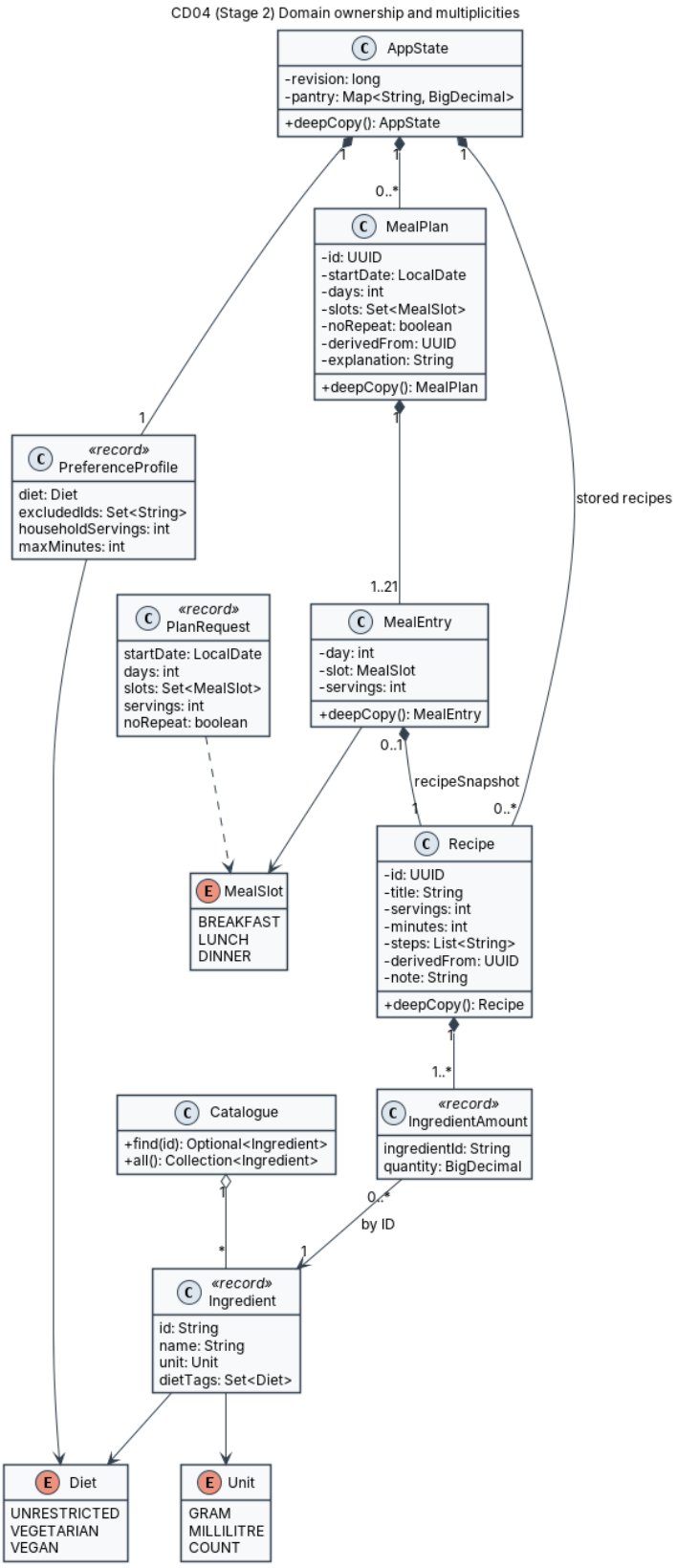


Figure 4. CD04: domain ownership and multiplicities.

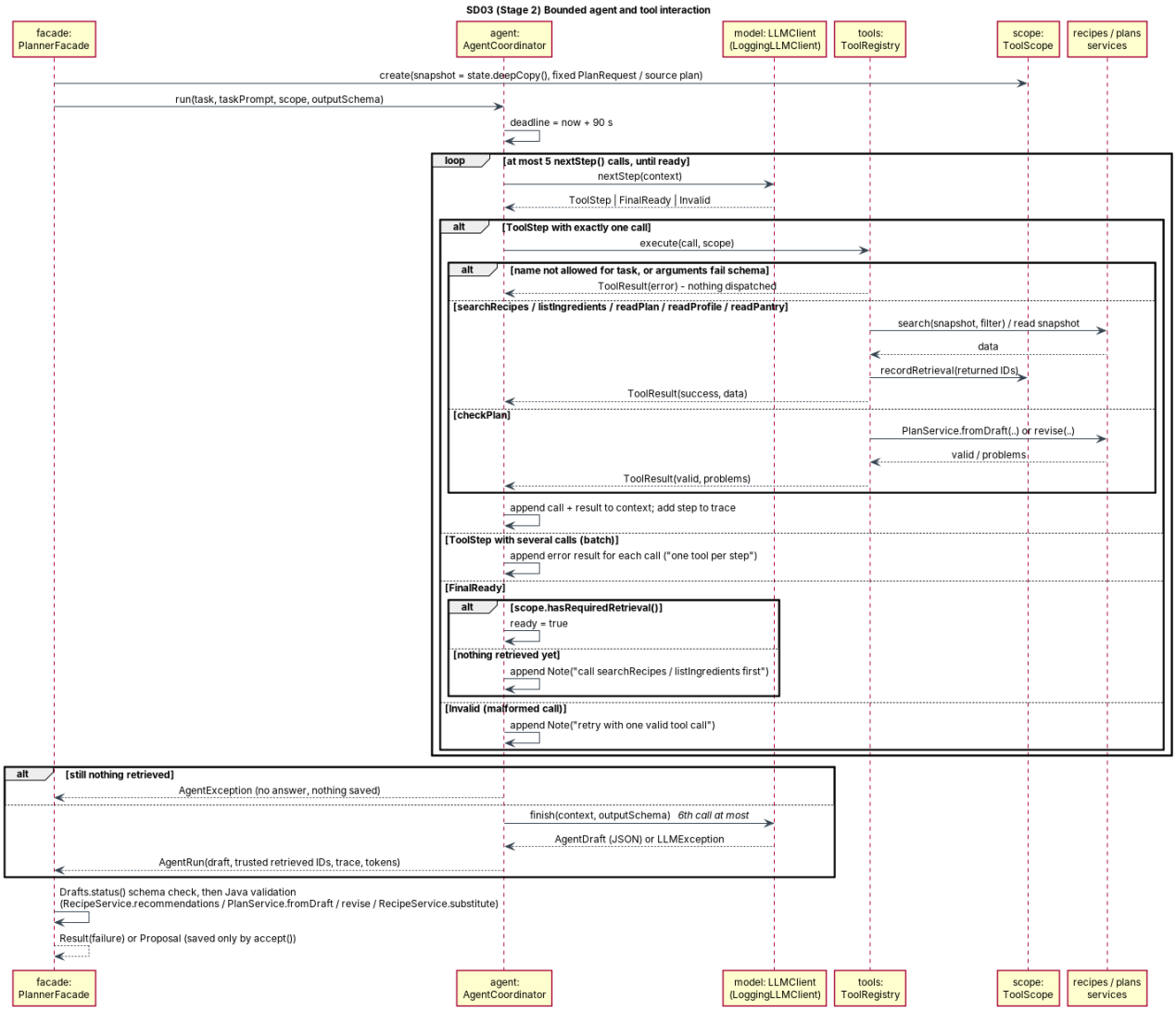


Figure 5. SD03 (updated): bounded agent and tool interaction.

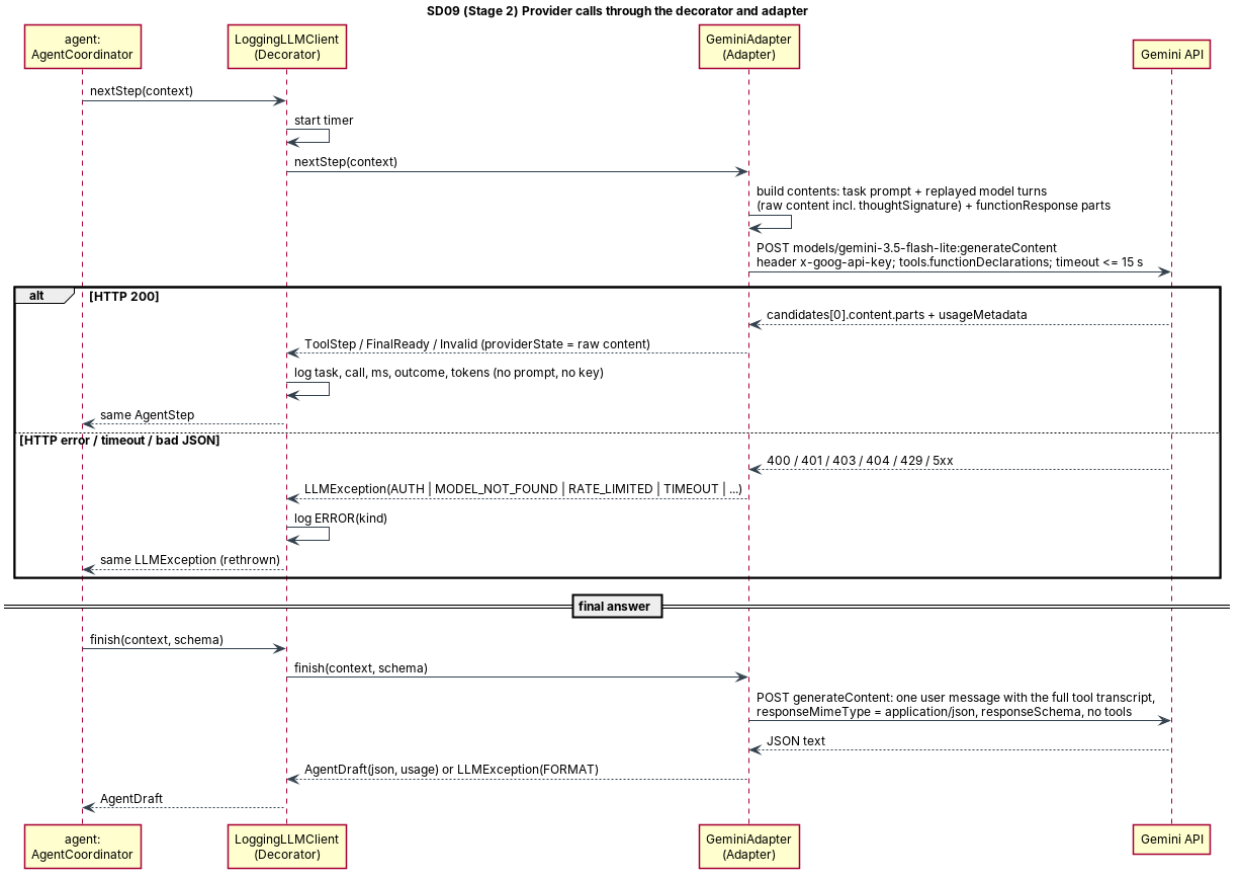


Figure 6. SD09 (updated): provider calls through the decorator and adapter.

4 Changes and rationale

No feature was added, removed or replaced, and all five patterns are the Stage 1 patterns. The table records the architecture and class-level changes, in the order Original design -> Implemented change -> Reason -> Impact.

Original design	Implemented change	Reason	Impact
Build tool not specified; "JUnit" testing.	Gradle wrapper, Jackson 2.16.1 for JSON, JUnit 4.13.2.	Reproducible build that needs only a JDK. These exact versions could be compiled and tested in the AI coding workspace, where Maven Central was blocked.	Build files and wrapper committed. Tests can move to JUnit 5 in Stage 3 if wanted.
API key from an environment variable.	`AppConfig` reads a Git-ignored `.env`; environment variables override it. `.env.example` holds a placeholder.	Stage 2 API-key rules; simpler for local setup.	New `AppConfig`; README explains setup. The key is only ever sent in a request header.
AgentStep = ToolCall or FinalReady.	Sealed `AgentStep`: `ToolStep` (a list, so batches can be rejected), `FinalReady`, `Invalid` (malformed provider call). Each carries an opaque `providerState` and `TokenUsage`.	Gemini 3 requires thought signatures to be replayed exactly. Malformed calls should be retried, not crash the run.	Provider details stay in the adapter (Adapter pattern preserved).
finish(context, schema) "with a structured schema yet without	`finish()` sends the full tool transcript as text in one message, with a response schema and no	Avoids replaying function-call history without tools while keeping the "no tools in the final	Same call budget (5 + 1).

Original design	Implemented change	Reason	Impact
tools".	tool declarations.	call" rule.	
Coordinator stops when FinalReady is received.	FinalReady is accepted only after the task's retrieval tool (`searchRecipes`, or `listIngredients` for F08) has run; otherwise a note asks the model to retrieve first. If 5 steps pass with retrieval done, `finish()` is still called; with no retrieval the run fails.	Enforces "IDs must come from tool results" before the answer, and handles budget exhaustion explicitly.	`ToolScope.hasRequiredRetrieval()`; tested in `AgentWorkflowTest`.
ToolRegistry.execute(name, args, state).	`execute(ToolCall, ToolScope)`. The new `ToolScope` holds the snapshot, the fixed `PlanRequest` or source plan, and the trusted retrieved IDs.	checkPlan needs the fixed request and the IDs retrieved so far; retrieved IDs must come from tools, not from the model.	New class `ToolScope`; same six tools.
Tool arguments as designed.	`searchRecipes` also accepts `excludeIngredientIds`. `readPlan` adds date, weekday and current eligibility for each meal.	Requests such as "no mushrooms" or "Tuesday dinner" must map to checkable data.	Tool results stay read-only.
F05: Java checks "supported request constraints".	The final answer includes `appliedConstraints` (maxMinutes, must-include, avoid). Java checks every recommendation against them.	Makes the model's reading of the request explicit and verifiable.	`RecommendationDraft`, `RecommendationList`.
F06 step 3: model returns recipe IDs and servings.	The model returns only day, slot and recipe ID. Servings always come from the `PlanRequest`.	Stage 1 also says the model cannot change servings; this removes the contradiction.	Simpler schema; the builder enforces servings.
Final answers: recommendations, plan entries, replacements, substitution.	Every answer also has `status` (OK, NO_MATCH, NEEDS_CLARIFICATION, UNSUPPORTED) and `message`.	Lets the agent report ambiguous or impossible requests honestly instead of inventing an answer.	Shown as a clear failure message; nothing saved.
Proposal assigns new object IDs at acceptance (SD08).	IDs are assigned when the preview is built; a rejected preview's ID is simply discarded.	The candidate state can be built once and saved unchanged.	No visible effect; aliases (R22, P02) appear on acceptance.
Implicit helper types.	Added `ProfileInput`, `PantryInput`, `RecipeDocument`, `SearchFilter`, `PlanDraft`, `RecommendationDraft`, `SubstitutionDraft`, `ShoppingList`, `DomainException`, `Result.details`, `AgentRun`, `AgentPrompts`, `Drafts`, `Schemas`, `TextFormat`, `AppContext`, `SeedData`, `DataDirectoryLock`, `ConnectionCheck`.	Concrete DTOs and helpers needed to implement the Stage 1 contracts.	Shown as <<new in Stage 2>> in CD01-CD03.
GUI described per feature.	One window with five tabs and a modal preview dialog; `submit()` runs every facade call on a `SwingWorker` and blocks input meanwhile.	Keeps the interface simple while meeting the background-execution rule.	No business logic in the GUI.
CLI examples in Stage 1.	All Stage 1 commands kept, plus `recipe list/show`, `plan list/show`, `ingredients`, `status` and `llm-check`.	Users need to see IDs and verify the provider.	Supporting commands only.
Provider failures are reported explicitly.	Still reported explicitly, but `GeminiAdapter` first retries once after HTTP 429 when Google's	Live testing on the free tier hit the per-minute quota during consecutive AI actions.	Fewer failed requests during normal use; persistent limits still give a

Original design	Implemented change	Reason	Impact
	suggested delay is at most 30 s and fits within the 90 s limit.		clear message and save nothing.
JsonStateStore.load(): AppState.	`load()` returns `Optional<AppState>`; first run seeds 21 recipes with stable IDs; invalid saved data stops start-up.	Distinguishes "no data yet" from "invalid data".	Matches the Stage 1 consistency rules.

5 AI tools and collaboration log

AI development tools were used as the course requires. I remain responsible for understanding, reviewing, integrating and validating all submitted code.

Tool / platform	LLM / model	Used for	Token cost	Money cost
Claude (Anthropic): Claude app agentic session with file and shell tools	Claude Opus 5.5	Reading the Stage 1/2 documents and building a requirements checklist; implementing all packages (domain, services, persistence, agent, Gemini adapter, CLI, GUI); JUnit tests; debugging; GUI screenshot checks; PlantUML diagrams; README; demo script; drafting this report	About 74.1 million tokens processed over 159 requests (72.67M cache reads, 1.08M cache writes, 360,775 output, 340 uncached input), from the Claude app's usage report on 8 Nov 2026 at 12:50; later work on the videos and report is not included	Subscription: Claude Ultra plan, CA$316.40 per month (invoice of 25 Oct 2026; the fee covers all use of the account, not only this project). API charges: $0 (no API organization; work used the subscription).
Gemini API (runtime model inside the application, not a development tool)	gemini-3.5-flash-lite	F05-F08 agent steps and final answers; llm-check	About 114,000 tokens over about 89 model calls in total, as counted by the app's LoggingLLMClient (live checks on 8 Nov: 22,466 input + 778 output; recording sessions: about 90,000 more)	$0 (Google AI Studio free tier, no billing account)

Subscription fees and API usage costs are listed separately. Token counts come from the Claude app usage report and from MealPlanner's own call log.

Representative interactions

1. Requirements and repository review. I gave Claude the Stage 2 instructions, the Stage 1 report and a written brief. Before writing code it reported that the repository had no commits in periods 1 and 2, so the deduction already applied and backdating was not an option. It also reported that its session could not push to my repository (we used Git bundles that I pulled and pushed myself), that the original Project Instructions PDF had not been supplied, and that `gemini-3.5-flash-lite` is a current stable model. It also found that Gemini 3 requires thought signatures to be returned during function calling.

2. Build environment. Maven Central was blocked in Claude's workspace. Claude chose a Gradle wrapper with the Jackson and JUnit versions it could compile and test, explained why, and verified every commit in a clean checkout before handing it over. The trade-off (JUnit 4 instead of 5) is recorded in Section 4.

3. Agent design decisions. While implementing SD03, Claude proposed three refinements, which are recorded in Section 4: reject a READY signal until candidates were retrieved, send the final call as a text transcript with a response schema and no tools, and make the model report the constraints it extracted so Java can check them. Each is covered by a test with a scripted model.

4. Local setup problems. Using my pasted terminal output, Claude found that my GitHub CLI token had no scopes (so pushing would fail) and guided me through a browser login with the `repo` scope. It also handled a bundle file that was not where the command expected it.

5. GUI and demo verification. Claude ran the real GUI on a virtual display, reviewed screenshots of every tab, and drove a generate-preview-accept-shopping-list flow. This found a recipe-detail pane that was too narrow and messages that were not wrapped, and both were fixed. Running the demo commands also showed that two planned demo scenarios did not behave as first written, so the script was corrected.

6. Live provider check and rate limiting. Before my first live test, Claude compared the adapter with Google's REST examples and changed the schemas to Gemini's uppercase type names. `llm-check` then passed on the first run. A live CLI run of F05-F07 succeeded, but F08 hit the free-tier per-minute limit (HTTP 429). The app reported this clearly and saved nothing. Claude added a single retry that waits for Google's suggested delay, and F08 then succeeded (milk replaced by soy milk, original recipe unchanged).

7. API key exposure handled. While configuring `.env` I accidentally typed my API key into a command that I pasted into the chat. Claude told me to treat the key as exposed, delete it in AI Studio and create a new one, and to put the new key only into the Git-ignored `.env` file. I did this before running any test. The key never entered the repository.

6 Development and verification summary

6.1 Weekly commit summary

Period	Dates (Toronto)	Meaningful code commits	Notes
1	Oct 20-26, 2026	0	No commits; minimum not met.
2	Oct 27-Nov 2, 2026	0	No commits; minimum not met.
3	Nov 3-9, 2026	10	Implementation commits listed below (pushed to GitHub on 8 November 2026).

Because periods 1 and 2 have no commits, the single 5-point deduction applies. No commits were backdated or created artificially. Each period-3 commit below changes implementation or test code, and each was checked to compile and pass its tests on its own.

#	Commit time (Toronto)	Description	Link
1	2026-11-06 20:40	Set up Gradle build and core domain model	e364e08
2	2026-11-06 20:40	Add JSON state store with atomic saves and data-directory lock	4f02948
3	2026-11-06 20:40	Add constraint validator, recipe/plan/shopping services and seed data	9e2e86a
4	2026-11-06 20:40	Add PlannerFacade with preview/accept flow for deterministic features	6394ac4
5	2026-11-06 20:46	Add LLMClient adapter for Gemini and sanitized logging decorator	efa3251
6	2026-11-06 20:52	Add bounded agent workflow with validated tools for F05-F08	b24e07d
7	2026-11-06 20:56	Add interactive CLI, local .env configuration and app startup	e222f69

#	Commit time (Toronto)	Description	Link
8	2026-11-06 21:01	Add Swing GUI with background execution and preview dialogs	aa40c57
9	2026-11-08 11:59	Translate tool and response schemas to Gemini's Schema type names	d6d5e0e
10	2026-11-08 12:22	Retry once after Gemini rate limiting when the suggested wait fits	931f7bd

6.2 Checks performed

- **JUnit (78 tests, all passing on each implementation commit):** domain copies and builder (7), JSON store, atomic save and lock (5), deterministic services, including 200 g x 5/2 = 500 g and 300 g + 250 g = 550 g (15), facade preview/accept/stale/failed-save paths (12), Gemini adapter against a local fake HTTP server, covering request format, thought-signature replay, key only in header, error mapping and timeouts (11), logging decorator (2), agent workflow with a scripted model, covering F05-F08, the step budget, timeout, unretrieved IDs, schema violations and impossible plans (16), CLI command flows (6), configuration and start-up (4).
- **Clean-checkout builds:** each of the 8 implementation commits was checked out into an empty worktree, built and tested on its own.
- **CLI end-to-end:** the built jar was run with piped commands for preferences, pantry, search, scale with accept, import with accept, invalid and malformed imports, a stale preview and a missing plan. A second process on the same data folder was refused, and the AI commands without a key reported the missing key while other features kept working.
- **GUI:** launched on a virtual X display. All five tabs rendered, field-level validation and the no-key message appeared, and a scripted-model plan was generated, previewed with its agent steps, accepted, saved to disk and turned into a shopping list. The totals were checked by hand.
- **Student machine (macOS, JDK 23):** `./gradlew build` succeeded, including all tests; `git check-ignore .env` confirmed the key file is ignored.
- **Live Gemini (8 Nov 2026):** `llm-check` passed. A live CLI session ran F05 (listIngredients and searchRecipes, then 1 validated recommendation with Java-computed shortages), F06 (search, `checkPlan` valid, 3 distinct dinners), F07 (readPlan, search, `checkPlan`; only Tuesday changed; saved as P02 derived from P01), F10 (totals for P02) and F08 (readProfile and listIngredients, then milk replaced by soy milk). The F08 run first hit HTTP 429 and succeeded after the retry fix.

6.3 Important defects found and corrected

- Gemini 3 rejects function-calling requests that do not return thought signatures. Found in Google's documentation before coding; the adapter replays the raw model content, and a test checks that the signature is sent back.
- Google's REST examples write schema types in uppercase (OBJECT, ARRAY, STRING), while the adapter sent lowercase. This was found by re-checking the official documentation before the first live call, and fixed by translating schemas in `GeminiAdapter` (commit d6d5e0e).
- Live testing on the free tier hit HTTP 429 after several AI actions in a row. The adapter now waits once for Google's suggested retry delay (up to 30 s, within the 90 s limit) and retries the request, reporting the limit clearly if it persists (commit 931f7bd).
- The CLI printed "Working with the AI agent" even when AI was unavailable. Fixed in the CLI commit.
- The GUI recipe-detail pane was too narrow and long messages were cut off. Found in screenshots and fixed in the GUI commit (divider position, word wrap).

- The shared test fixture depended on every layer, so early commits would not compile on their own. Split into `TestRecipes` and `Fixtures`.
- Generating the Gradle wrapper failed in the AI workspace because the distribution URL could not be checked. It was generated without the check, and validation was re-enabled in the committed wrapper properties.
- Two demo scenarios (pantry-only search, vegan substitution) did not behave as first written when run. The script was corrected to match the real behaviour.
- The GitHub CLI token had no scopes, so pushes would fail. Re-authenticated through the browser with the `repo` scope.

6.4 Remaining limitations and Stage 3 work

- Stage 3: full deterministic test suite and KUMA agent-behaviour validation against the live model.
- Live AI answers can vary between runs. A weak answer is rejected by Java validation with an explanation, and the user can retry. On the free tier, several AI actions within a minute can still hit the rate limit after the automatic retry.
- No recipe or plan deletion, no in-place plan editing, no unit conversion, nutrition or prices (Stage 1 scope). Scaled quantities are stored at full DECIMAL128 precision by design and rounded only for display.
- Single local user. Only one GUI or CLI process can use a data folder at a time.

7 Demonstration index

Video 1 shows the GUI and Video 2 shows the CLI. Together they demonstrate every feature with its input, output, AI involvement and relevant error behaviour. Each link opens the video at the feature's segment, without an access request.

Error and alternative behaviour shown in the videos includes: invalid preference and pantry input, invalid and malformed recipe imports, an unsupported AI request (calories) refused with an explanation, pantry-only search with no results, out-of-range servings, an impossible no-repeat plan stopped in Java before any AI call, a mistyped CLI option rejected, a non-existent plan, the missing-key mode, and an AI substitution rejected by Java because the result would not be vegan. Stale-preview rejection was not recorded; it is covered by `FacadeDeterministicTest.staleProposalIsRejected`.

Feature	GUI (Video 1)	CLI (Video 2)
F01 Manage dietary preferences	Video 1 (GUI) 0:00	Video 2 (CLI) 0:46
F02 Manage pantry inventory	Video 1 (GUI) 1:10	Video 2 (CLI) 1:00
F03 Create and import structured recipes	Video 1 (GUI) 2:00	Video 2 (CLI) 1:10
F04 Search and filter recipes	Video 1 (GUI) 4:18	Video 2 (CLI) 1:30
F05 Recommend recipes (AI)	Video 1 (GUI) 3:08	Video 2 (CLI) 1:44
F06 Generate a meal plan (AI)	Video 1 (GUI) 6:46	Video 2 (CLI) 2:30
F07 Revise a plan (AI)	Video 1 (GUI) 8:02	Video 2 (CLI) 3:06
F08 Ingredient substitution (AI)	Video 1 (GUI) 5:58	Video 2 (CLI) 3:56
F09 Adjust serving quantities	Video 1 (GUI) 5:14	Video 2 (CLI) 1:54

Feature	GUI (Video 1)	CLI (Video 2)
F10 Consolidated shopping list	Video 1 (GUI) 7:46	Video 2 (CLI) 2:54
Live provider check (llm-check)	-	Video 2 (CLI) 0:00
Missing API key: AI refused, other features work	-	Video 2 (CLI) 3:34
Restart: saved data persists	Video 1 (GUI) 8:56	-

References

1. EECS3311 Fall 2026, Course Project Stage 2 Instructions (supplied PDF).
2. S. Kavas, MealPlanner Stage 1 Project Design Report, 2 October 2026 (`MealPlanner_Stage1_Design.docx`).
3. Google AI for Developers, Gemini 3.5 Flash-Lite model page: ai.google.dev/gemini-api/docs/models/gemini-3.5-flash-lite. Accessed 6 November 2026.
4. Google AI for Developers, Thought signatures: ai.google.dev/gemini-api/docs/thought-signatures. Accessed 6 November 2026.
5. S. Wang, EECS3311 lecture slides on creational and structural design patterns (Builder, Prototype, Adapter, Facade, Decorator).